

Probabilistic Programming

Marius Popescu

popescunmarius@gmail.com

2025 - 2026

PyMC Framework

Introductory Overview of PyMC

PyMC Framework

- Probabilistic Programming (PP) allows flexible specification of Bayesian statistical models in code.
- PyMC is a PP framework with an intuitive and readable, yet powerful, syntax that is close to the natural syntax statisticians use to describe models. PyMC allows for model specification in Python code, rather than in a domain-specific language, making it easy to learn, customize, and debug.
- Once we have defined our model, we have to perform inference to approximate the posterior distribution. PyMC supports two broad classes of inference: sampling and variational inference:
 - PyMC features next-generation Markov chain Monte Carlo (MCMC) sampling algorithms such as the No-U-Turn Sampler (NUTS), a self-tuning variant of Hamiltonian Monte Carlo (HMC). This class of samplers works well on high-dimensional and complex posterior distributions and allows many complex models to be fit without specialized knowledge about fitting algorithms.
 - PyMC supports various Variational Inference techniques. While these methods are much faster, they are often also less accurate and can lead to biased inference.

Model Specification

Model Creation

Models in PyMC are centered around the `Model` class. It has references to all random variables (RVs) and computes the model logp and its gradients. Usually, you would instantiate it as part of a `with` context:

```
with pm.Model() as model:
    mu = pm.Normal("mu", mu=0, sigma=1)
    obs = pm.Normal("obs", mu=mu, sigma=1, observed=rng.standard_normal(100))
```

```
model.basic_RVs          # [Output]: [mu ~ N(0, 1), obs ~ N(mu, 1)]
model.free_RVs           # [Output]: [mu ~ N(0, 1)]
model.observed_RVs       # [Output]: [obs ~ N(mu, 1)]
model.compile_logp()({ "mu": 0 }) # [Output]: array(-143.03962875)
```

Two Types of Variables

- Bayesian *random variables* have not necessarily arisen from a physical random process. The Bayesian interpretation of probability is epistemic, meaning random variable Z 's probability distribution $P(Z)$ represents our knowledge and uncertainty about Z 's value
- PyMC is concerned with two types of programming (random) variables: *stochastic* and *deterministic*
 - *Stochastic* variables are variables that are not deterministic, i.e., even if you knew all the values of the variables' parameters and components, it would still be random. Included in this category are instances of classes `Poisson`, `DiscreteUniform`, and `Exponential`
 - *Deterministic* variables are variables that are not random if the variables' parameters and components were known, but it is random by nature:

```
with pm.Model():
```

```
    x = pm.Normal("x", mu=0, sigma=1)
```

```
    y = pm.Gamma("y", alpha=1, beta=1)
```

```
    summed = x + y # variable summed is deterministic, but its value is random. What distribution?
```

Probability Distributions

Every probabilistic program consists of observed and unobserved Random Variables (RVs). Observed RVs are defined via likelihood distributions, while unobserved RVs are defined via prior distributions. In the PyMC module, the structure for probability distributions looks like this:

Distributions

- `pymc:api_distributions_continuous`
- `pymc:api_distributions_discrete`
- `pymc:api_distributions_multivariate`
- `pymc:api_distributions_mixture`
- `pymc:api_distributions_timeseries`
- `pymc:api_distributions_truncated`
- `pymc:api_distributions_censored`
- `pymc:api_distributions_simulator`
- `pymc:api_distributions_transformations`
- `pymc:api_distributions_utilities` (define custom distribution)

Bayes' Theorem (Memento)

$$P(A, B) = P(A|B)P(B) = P(B|A)P(A)$$

⇓

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$



1702 - 1761

Bayesian Modeling (Memento)

D = observed data

θ = model parameters

Likelihood

Prior Distribution

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

$$P(\theta|D) \propto P(D|\theta)P(\theta)$$

Posterior Distribution

Unobserved Random Variables

Every unobserved RV has the following calling signature: name (str), parameter keyword arguments. Thus, a normal prior can be defined in a model context like this:

```
with pm.Model():  
    x = pm.Normal("x", mu=0, sigma=1)
```

As with the model, we can evaluate its logp:

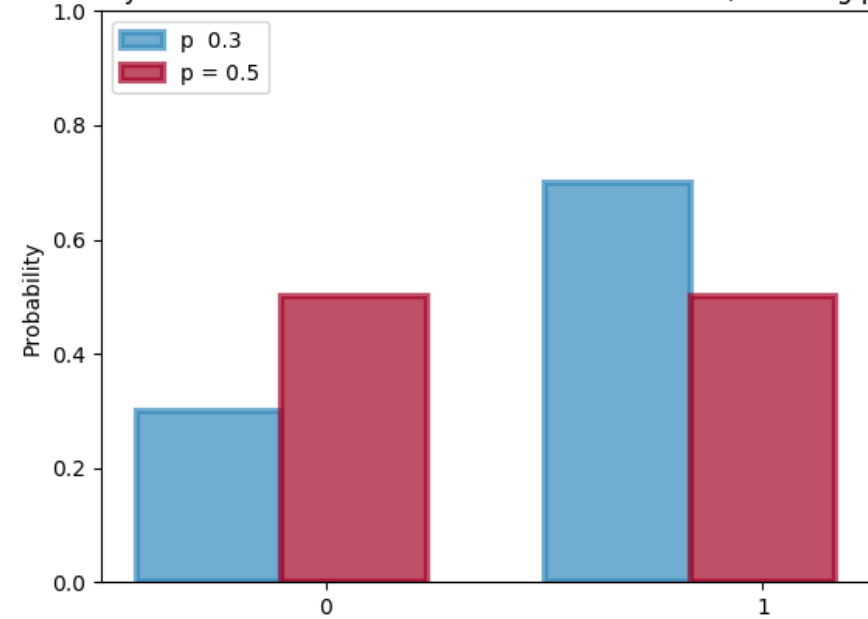
```
pm.logp(x, 0).eval() # [Output]: array(-0.91893853)
```

PyMC provides a large suite of built-in probability distributions

Bernoulli Distribution

$$Z \sim \text{Ber}(p)$$
$$P(Z = k) = p^k(1 - p)^{1-k}, \quad k = 0, 1$$
$$E(Z|p) = \sum_{k=0}^1 kP(Z = k) = p$$

Probability mass function of a Bernoulli random variable; differing p values

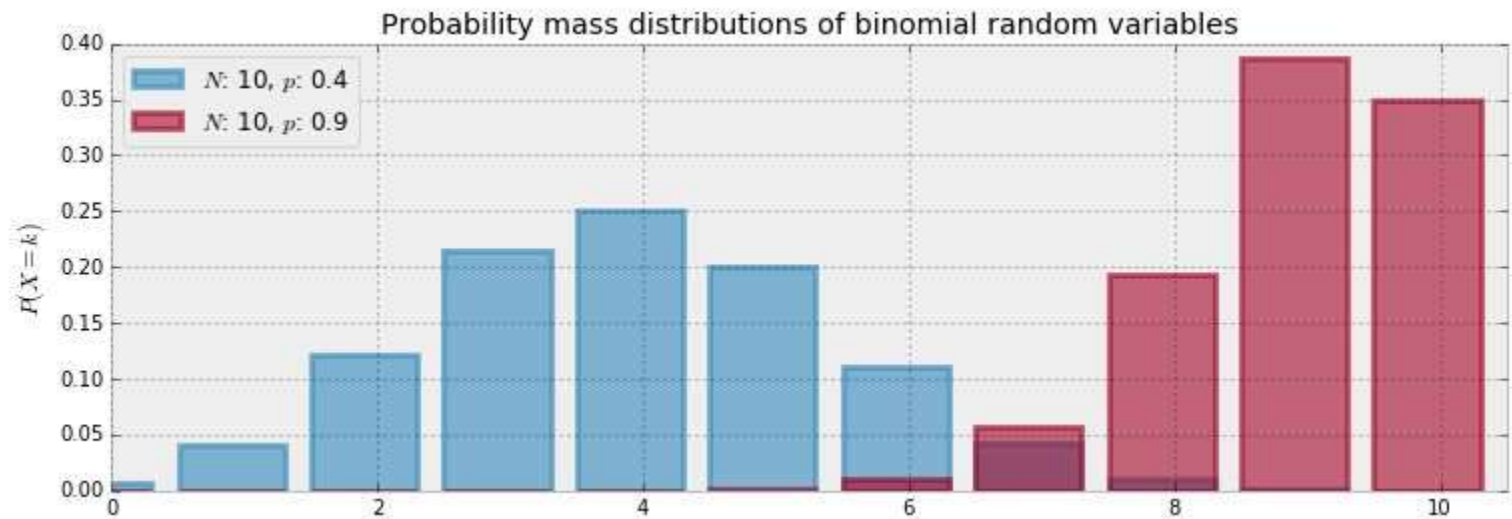


`Z = pymc.Bernoulli("Z", p=p)`

Binomial Distribution

$$Z \sim \text{Bin}(N, p)$$
$$P(Z = k) = \binom{N}{k} p^k (1 - p)^{N-k}, \quad k = 0, 1, \dots, N$$
$$E(Z|N, p) = \sum_{k=0}^N k P(Z = k) = Np$$

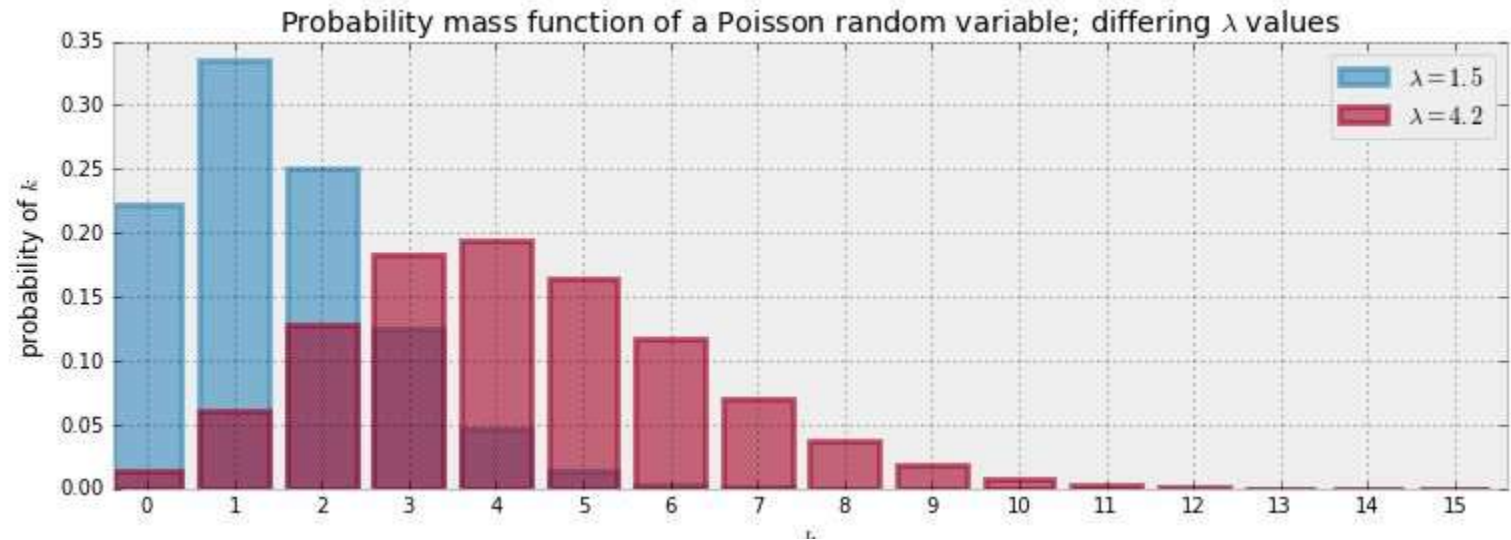
Z is the number of events that occurred in the N trials, and p is the probability of a single event



```
Z = pymc.Binomial("Z", n=N, p=p)
```

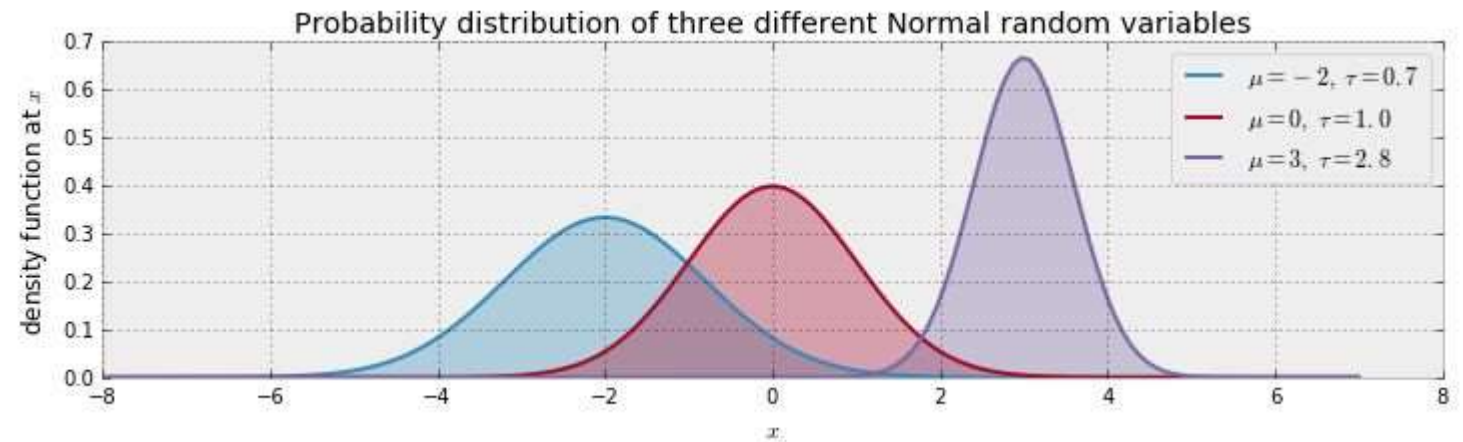
Poisson Distribution

$$Z \sim \text{Poi}(\lambda)$$
$$P(Z = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots, \quad \lambda \in \mathbb{R}_{>0}$$
$$E(Z|\lambda) = \sum_{k=0}^{\infty} kP(Z = k) = \lambda$$



```
Z=pymc.Poisson("Z", mu=lambda_)
```

Normal Distribution



$$Z \sim N\left(\mu, \frac{1}{\tau}\right)$$

τ (precision of the distribution) corresponds to $\frac{1}{\sigma^2}$

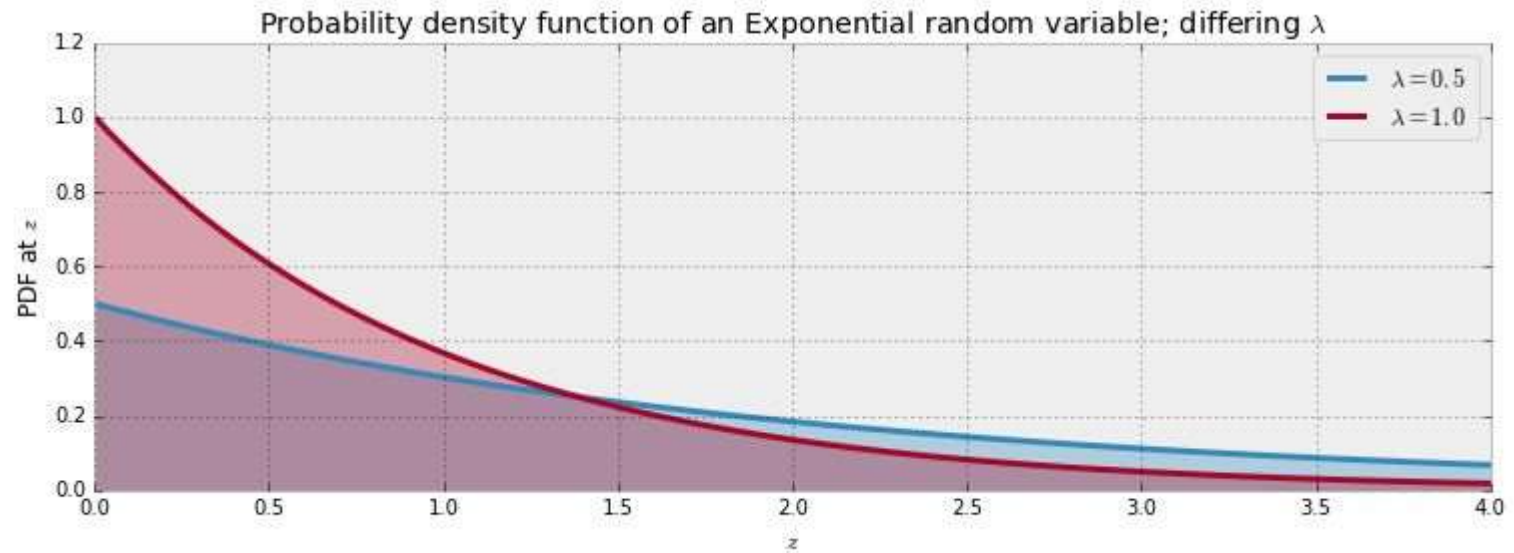
$$f_Z(z|\mu, \tau) = \sqrt{\frac{\tau}{2\pi}} e^{-\frac{\tau}{2}(z-\mu)^2}, \quad \tau > 0$$

$$E(Z|\mu, \tau) = \int_{-\infty}^{\infty} z f_Z(z|\mu, \tau) dz = \mu$$

```
Z=pymc.Normal("Z", mu, sigma|tau)
```

Exponential Distribution

$$Z \sim \text{Exp}(\lambda)$$
$$f_Z(z|\lambda) = \lambda e^{-\lambda z}, \quad z \geq 0$$
$$E(Z|\lambda) = \int_{-\infty}^{\infty} z f_Z(z|\lambda) dz = \frac{1}{\lambda}$$

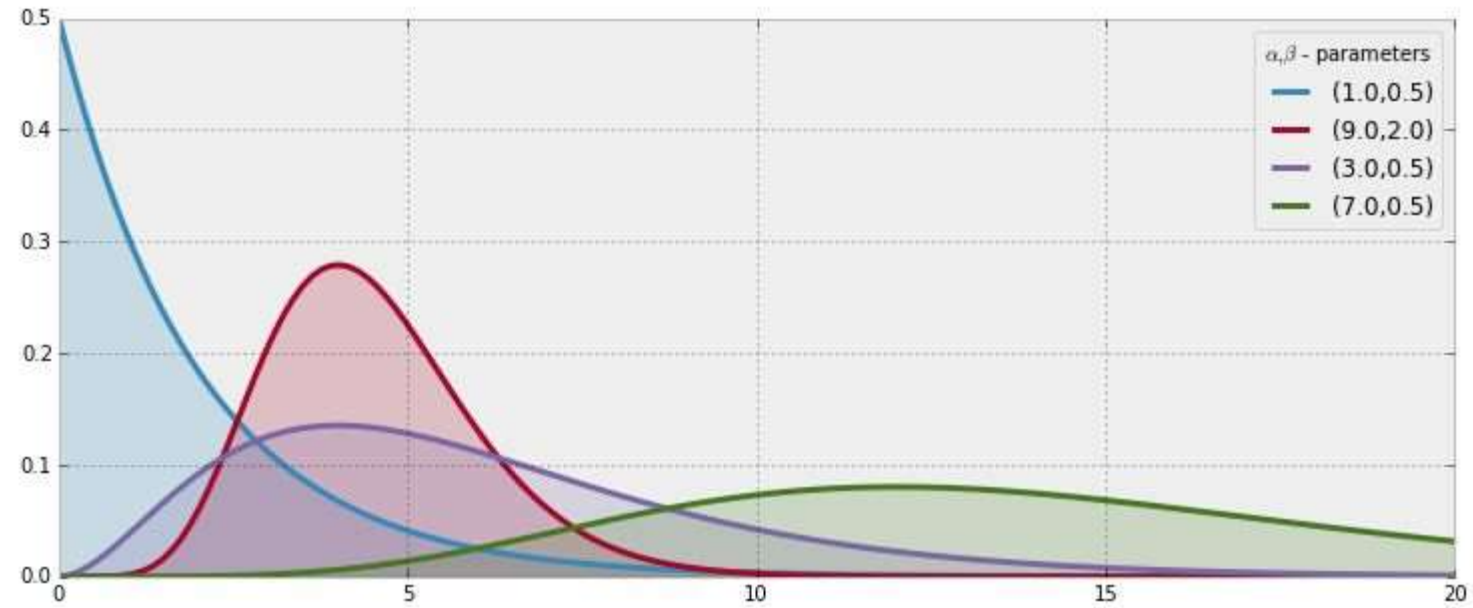


```
Z=pymc.Exponential("Z", lam=lambda_)
```


Gamma Distribution

$$Z \sim \text{Gamma}(\alpha, \beta)$$
$$f_Z(z|\alpha, \beta) = \frac{\beta^\alpha z^{\alpha-1} e^{-\beta z}}{\Gamma(\alpha)}, \quad \alpha > 0, \beta > 0, z \geq 0$$

$$E(Z|\alpha, \beta) = \int_{-\infty}^{\infty} z f_Z(z|\alpha, \beta) dz = \frac{\alpha}{\beta}$$



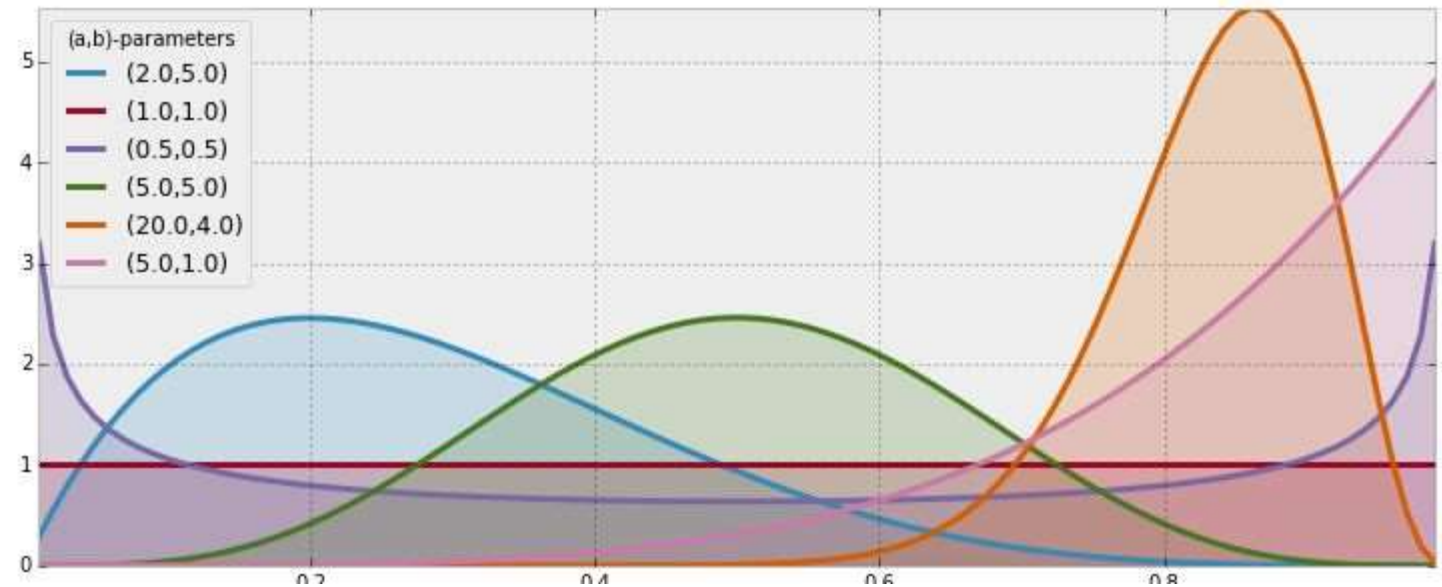
`Z=pymc.Gamma("Z", alpha, beta)`

Beta Distribution

$$Z \sim \text{Beta}(\alpha, \beta)$$

$$f_Z(z|\alpha, \beta) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} z^{\alpha-1} (1-z)^{\beta-1}, \quad \alpha > 0, \beta > 0, 0 < z < 1$$

$$E(Z|\alpha, \beta) = \int_{-\infty}^{\infty} z f_Z(z|\alpha, \beta) dz = \frac{\alpha}{\alpha + \beta}$$



`Z=pymc.Beta("Z", alpha, beta)`

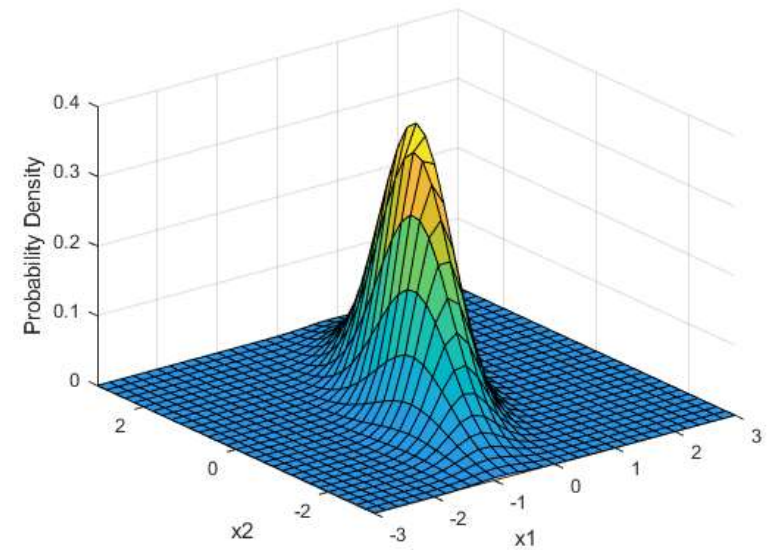
Multivariate Normal Distribution

$$Z \sim N(\mu, T)$$

T (precision of the distribution) is the inverse of the covariance matrix Σ

$$f_Z(z|\mu, \tau) = \sqrt{\frac{|T|}{2\pi}} e^{-\frac{1}{2}(z-\mu)^T T (z-\mu)}$$

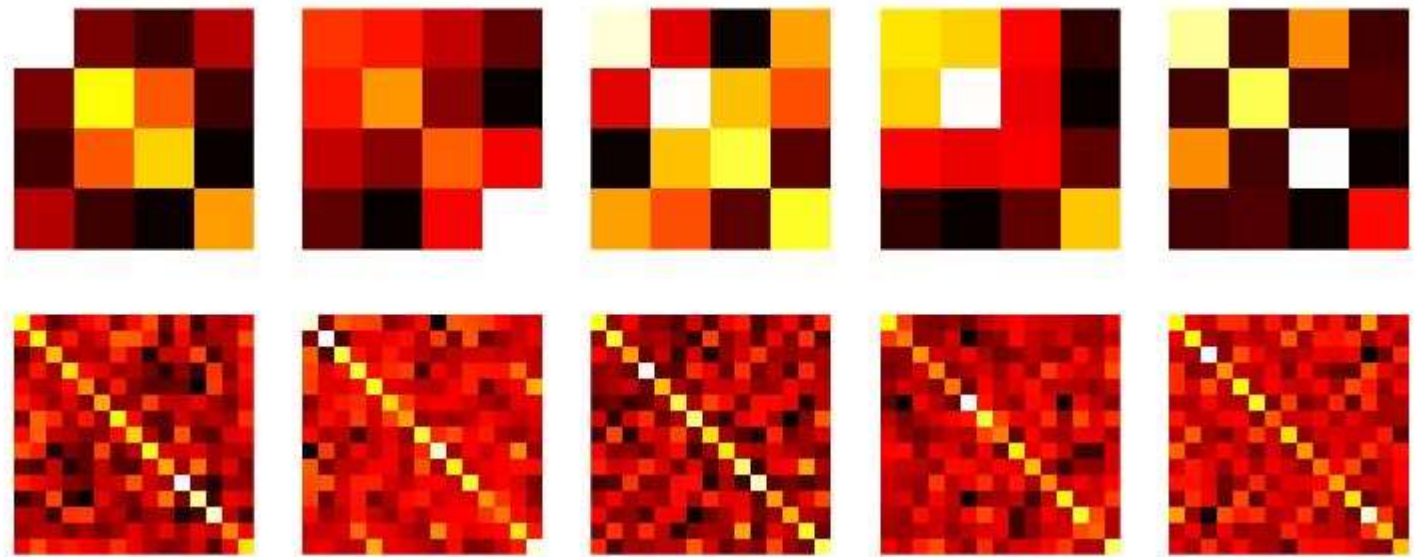
```
Z = pm.MvNormal("Z", mu=np.array([2.,4., 6.]), cov=np.identity(3))
```



Wishart Distribution

- Wishart distribution is a distribution over all positive semi-definite matrices
- Covariance matrices are positive-definite, hence the Wishart is an appropriate prior for covariance matrices

Random matrices from a Wishart Distribution



There Are More

<https://www.pymc.io/projects/docs/en/stable/api/distributions.html>



5.9.0

[Home](#)[Examples](#)[Learn](#)[API](#)[Community](#)[Contributing](#)

Section Navigation

Distributions

[Continuous](#)[Discrete](#)[Multivariate](#)[Mixture](#)[Timeseries](#)[Truncated](#)[Censored](#)[Simulator](#)[Transformations](#)

Discrete

[pymc.Bernoulli](#)[pymc.BetaBinomial](#)[pymc.Binomial](#)[pymc.Categorical](#)[pymc.DiscreteUniform](#)[pymc.DiscreteWeibull](#)[pymc.Geometric](#)[pymc.HyperGeometric](#)[pymc.NegativeBinomial](#)[pymc.OrderedLogistic](#)[pymc.OrderedProbit](#)[pymc.Poisson](#)



5.9.0

[Home](#)[Examples](#)[Learn](#)[API](#)[Community](#)[Contributing](#)

Section Navigation

Distributions

[Continuous](#)[Discrete](#)[Multivariate](#)[Mixture](#)[Timeseries](#)[Truncated](#)[Censored](#)[Simulator](#)[Transformations](#)[Distribution utilities](#)[Gaussian Processes](#)[Model](#)[Examples](#)[Read The Docs](#)

v: stable

Multivariate

[pymc.CAR](#)[pymc.Dirichlet](#)[pymc.DirichletMultinomial](#)[pymc.ICAR](#)[pymc.KroneckerNormal](#)[pymc.LKJCholeskyCov](#)[pymc.LKJCorr](#)[pymc.MatrixNormal](#)[pymc.Multinomial](#)[pymc.MvNormal](#)[pymc.MvStudentT](#)[pymc.OrderedMultinomial](#)[pymc.StickBreakingWeights](#)[pymc.Wishart](#)[pymc.WishartBartlett](#)[pymc.ZeroSumNormal](#)

Observed Random Variables

Observed RVs are defined just like unobserved RVs but require data to be passed into the `observed` keyword argument:

```
with pm.Model():  
    obs = pm.Normal("x", mu=0, sigma=1, observed=rng.standard_normal(100))
```

`observed` supports lists, `numpy.ndarray` and `pytensor` data structures.

Lists of RVs / higher-dimensional RVs

In many models, we want multiple RVs. Users will sometimes try to create lists of RVs, like this:

```
with pm.Model():
    # bad:
    x = [pm.Normal(f"x_{i}", mu=0, sigma=1) for i in range(10)]
```

This works, but it is slow and not recommended. Instead, we can use coordinates:

```
coords = {"cities": ["Santiago", "Mumbai", "Tokyo"]}
with pm.Model(coords=coords) as model:
    # good:
    x = pm.Normal("x", mu=0, sigma=1, dims="cities")
```

`x` is now a array of length 3 and each of the 3 variables within this array is associated with a label. This will make it very easy to distinguish the 3 different variables when we go to look at results. We can index into this array or do linear algebra operations on it:

```
with model:
    y = x[0] * x[1] # indexing is supported
    x.dot(x.T) # linear algebra is supported
```

Initialize Random Variables

Though PyMC automatically initializes models, it is sometimes helpful to define initial values for RVs. This can be done via the `initval` kwarg:

```
with pm.Model(coords={"idx": np.arange(5)}) as model:
```

```
    x = pm.Normal("x", mu=0, sigma=1, dims="idx")
```

```
model.initial_point() # [Output]: {'x': array([0., 0., 0., 0., 0.])}
```

```
with pm.Model(coords={"idx": np.arange(5)}) as model:
```

```
    x = pm.Normal("x", mu=0, sigma=1, dims="idx", initval=rng.standard_normal(5))
```

```
model.initial_point() # [Output]: {'x': array([-0.36012097, -0.16168135,  1.07485641, -0.08855632, -0.03857412])}
```

Custom Distributions

`pymc.CustomDist` is a helper class to create custom distributions

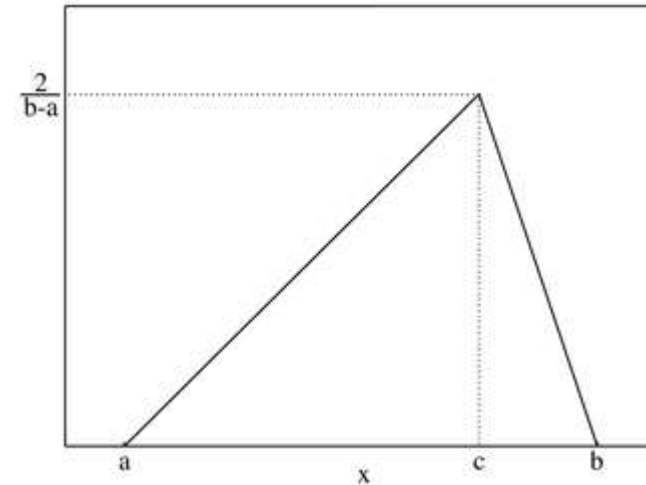
A user can provide a `random` function that returns numerical draws (e.g., via NumPy routines), and a `logp` function that must return a Python graph that represents the logp graph when evaluated. This is used for mcmc sampling.

Example: Triangular Distribution

- The *triangular distribution* is a continuous probability distribution with lower limit a , upper limit b and mode c , where $a < b$ and $a \leq c \leq b$
- The probability density function:

$$f(z) = \begin{cases} 0 & \text{for } z < a \\ \frac{2(z-a)}{(b-a)(c-a)} & \text{for } a \leq z < c \\ \frac{2}{b-a} & \text{for } z = c \\ \frac{2(b-z)}{(b-a)(b-c)} & \text{for } c < z \leq b \\ 0 & \text{for } b < z \end{cases}$$

- The triangular distribution is often used in ill-defined problems where the underlying distribution is not known, but some knowledge of the limits and mode exists



Example: Triangular Distribution

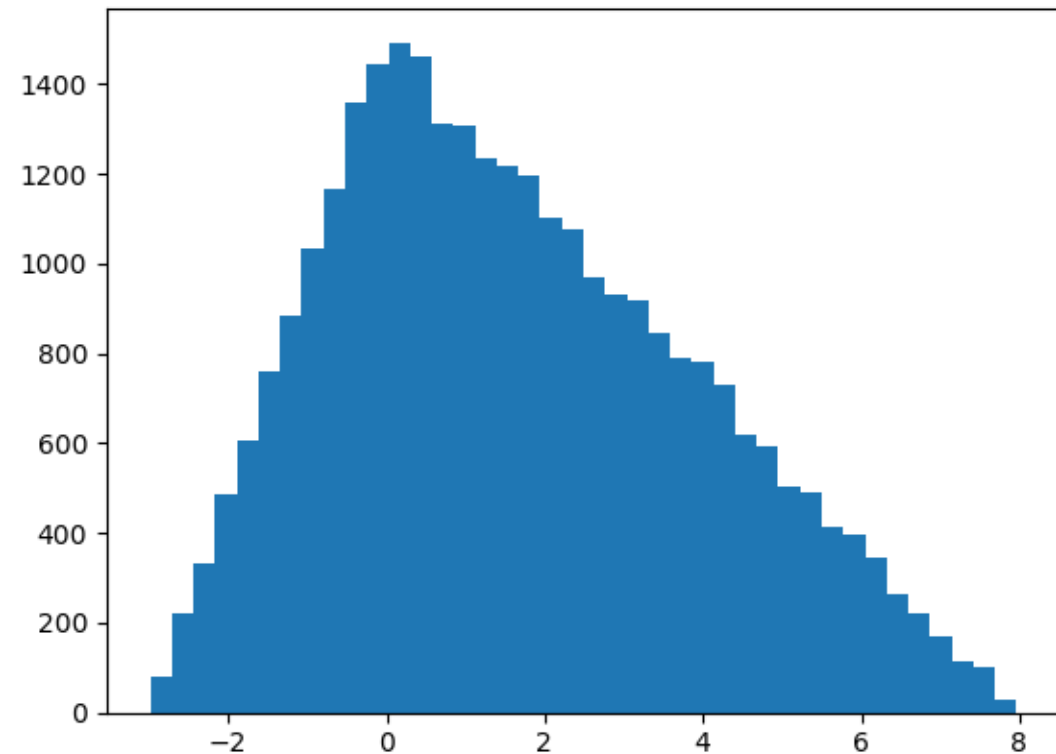
```
def triangular_logp(value, a, b, c):  
    if a <= value < c:  
        return np.log((2. * (value - a)) / ((b - a) * (c - a)))  
    if value == c:  
        return np.log(2. / (b - a))  
    if c < value <= b:  
        return np.log((2. * (b - value)) / ((b - a) * (b - c)))  
    return -np.inf  
  
def triangular_random(a, b, c, rng=None, size=None):  
    return np.random.triangular(a, c, b, size)  
  
with pm.Model() as model:  
    Z = pm.CustomDist(  
        'Z',  
        -3, 8, 0,  
        logp=triangular_logp,  
        random=triangular_random,  
    )
```

Example: Triangular Distribution

```
Z_samples = pm.draw(Z, draws=40000)
```

```
plt.hist(Z_samples, bins = 40)
```

```
plt.show()
```



PyMC Deterministic Variables

Deterministic variables

Deterministic variables are variables that are not random if the variables' parameters and components were known, but they are random by nature:

```
with pm.Model():  
    x = pm.Normal("x", mu=0, sigma=1)  
    y = pm.Gamma("y", alpha=1, beta=1)  
    summed = x + y # variable summed is deterministic, but its value is random. What distribution?
```

Deterministic variables

PyMC allows you to freely do algebra with RVs in all kinds of ways:

```
with pm.Model():  
    x = pm.Normal("x", mu=0, sigma=1)  
    y = pm.Gamma("y", alpha=1, beta=1)  
    plus_2 = x + 2  
    summed = x + y  
    squared = x**2  
    sined = pm.math.sin(x)
```

The expression that we are making
must be compatible with pytensor

Though these transformations work seamlessly, their results are not stored automatically. Thus, if you want to keep track of a transformed variable (`trace`), you have to use `pm.Deterministic`:

```
with pm.Model():  
    x = pm.Normal("x", mu=0, sigma=1)  
    plus_2 = pm.Deterministic("x plus 2", x + 2)
```

Note that `plus_2` can be used in the identical way to above, we only tell PyMC to keep track of this RV for us.

Not covered yet

Next Lectures

Distributions — PyMC 5.9.0 docs

pymc.io/projects/docs/en/stable/api/distributions.h...

PyMC 5.9.0 Home Examples Learn API Community Contributing

Distributions

- Continuous
- Discrete
- Multivariate
- Mixture
- Timeseries
- Truncated
- Censored
- Simulator
- Transformations
- Distribution utilities
- Gaussian Processes
- Model
- Samplers
- Variational Inference
- Sequential Monte Carlo
- Data
- Ordinary differential equations (ODEs)
- Probability
- Tuning
- Math
- PyTensor utils
- shape_utils
- Storage backends
- Other utils

API > Distributions

Distributions

Continuous

- pymc.AsymmetricLaplace
- pymc.Beta
- pymc.Cauchy
- pymc.ChiSquared
- pymc.ExGaussian
- pymc.Exponential
- pymc.Flat
- pymc.Gamma
- pymc.Gumbel
- pymc.HalfCauchy
- pymc.HalfFlat
- pymc.HalfNormal
- pymc.HalfStudentT
- pymc.Interpolated
- pymc.InverseGamma
- pymc.Kumaraswamy
- pymc.Laplace
- pymc.Logistic
- pymc.LogitNormal
- pymc.LogNormal
- pymc.Moyal
- pymc.Normal

Read The Docs v: stable

Example: How to Conduct Embarrassing Polls

Cheating among Students

Differential Privacy

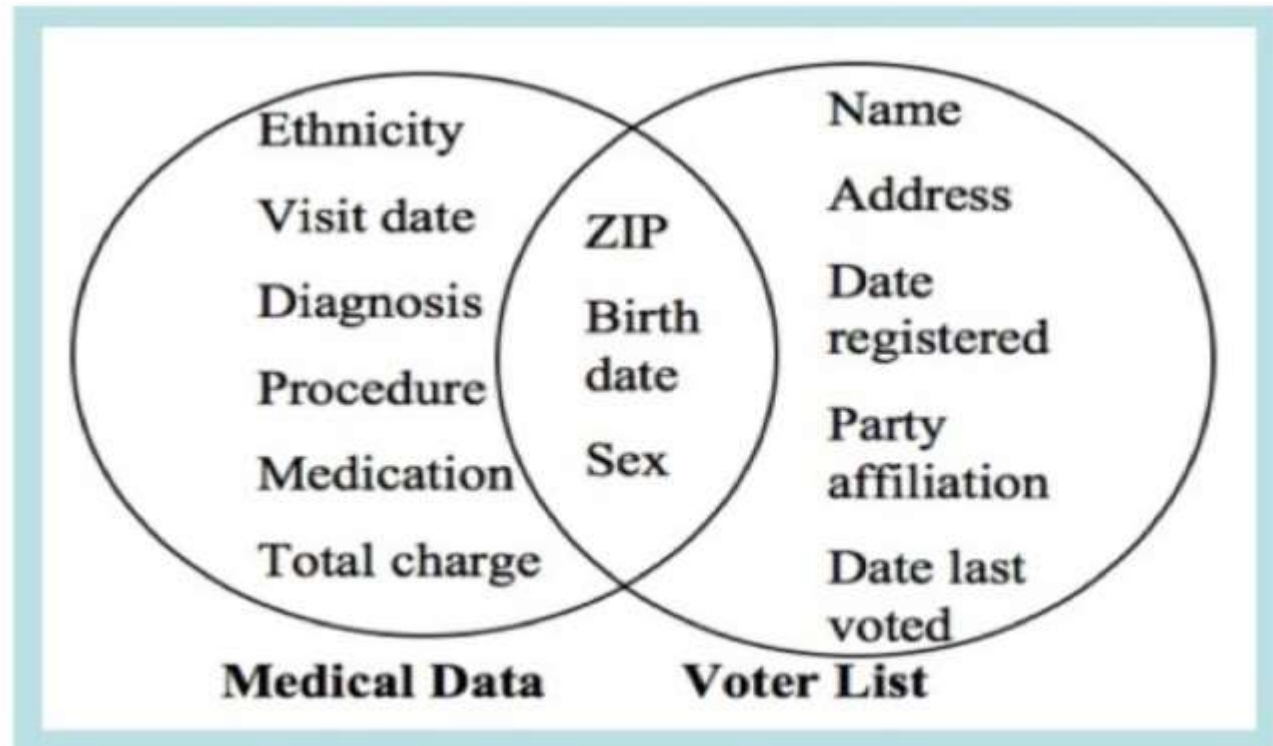
“ANONYMIZED DATA ISN'T”



William Weld vs Latanya Sweeney

Massachusetts Group Insurance Commission (1997):
Anonymized medical history of state employees (all
hospital visits, diagnosis, prescriptions)

Latanya Sweeney (MIT grad student): \$20 – Cambridge voter roll



Samarati, Pierangela; Sweeney, Latanya (1998). "Protecting privacy when disclosing information: k-anonymity and its enforcement through generalization and suppression". Harvard Data Privacy Lab.

L. Sweeney. k-anonymity: a model for protecting privacy. International Journal on Uncertainty, Fuzziness and Knowledge-based Systems, 10 1998, 2002; 557-570.

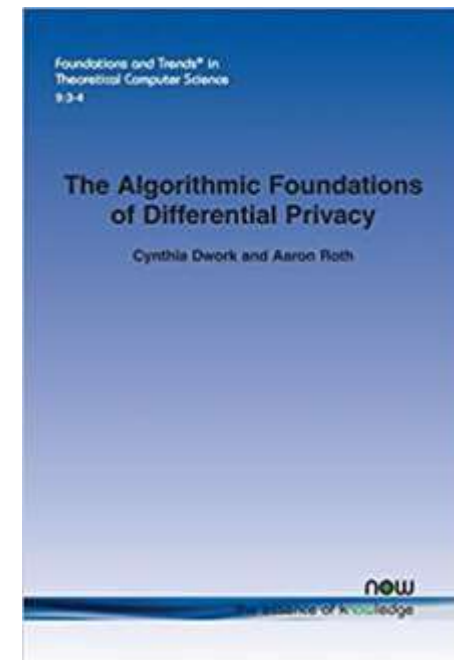
Learning and Privacy

- To do machine learning, we need data
- What if the data contains sensitive information?
 - medical data, web search query data, salary data, student grade data
- Even if the (person running the) learning algorithm can be trusted, perhaps the output of the algorithm reveals sensitive info
- E.g., SVM or perceptron on medical data:
 - Suppose feature j is has-green-hair and the learned w has $w_j \neq 0$
 - If there is only one person in town with green hair, you know they were in the study

Differential Privacy

Differential privacy requires that adding or removing the data record of a single individual not change the probability of any outcome by “much”. It requires this of an algorithm even in the worst case, no matter what records the other individuals have provided and no matter how unusual the added or removed data is.

It guarantees that no matter what your data is, and no matter what thing you are concerned about occurring because of the use of your data, that thing becomes (almost) no more likely if you allow your data to be included in the study, compared to if you do not.



Differential Privacy

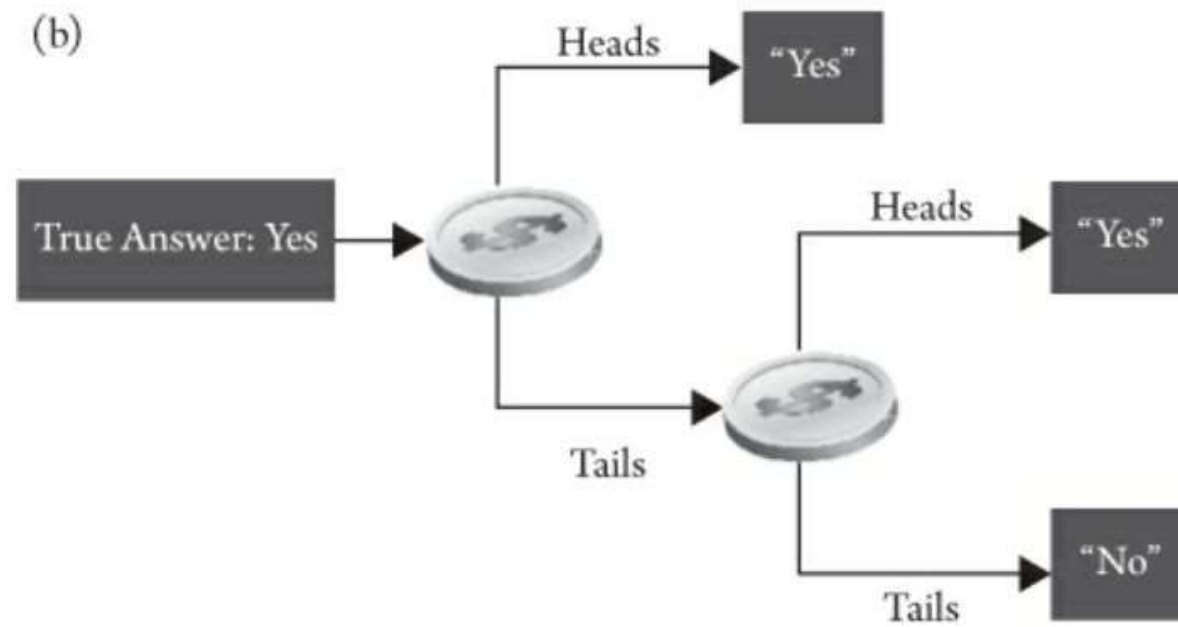
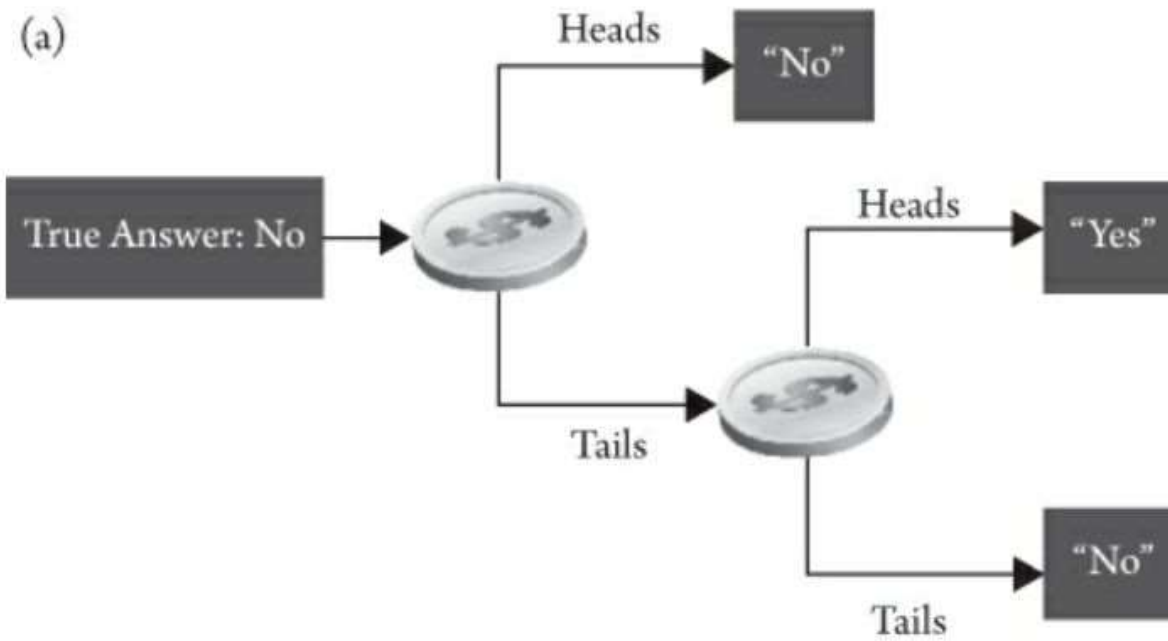
Differential privacy is a mathematical formalization of the foregoing idea—that we should be comparing what someone might learn from an analysis if any particular person's data was included in the dataset with what someone might learn if it was not.

Differential privacy uses randomness to deliberately add noise to computations, in a way that promises that any one person's data cannot be reverse-engineered from the results.

Privacy Algorithm

In the interview process for each student, the student flips a coin, hidden from the interviewer. The student agrees to answer honestly if the coin comes up heads. Otherwise, if the coin comes up tails, the student (secretly) flips the coin again, and answers "Yes, I did cheat" if the coin flip lands heads, and "No, I did not cheat", if the coin flip lands tails. This way, the interviewer does not know if a "Yes" was the result of a guilty plea, or a Heads on a second coin toss. Thus privacy is preserved and the researchers receive honest answers.

Warner, S. L. (March 1965). "Randomised response: a survey technique for eliminating evasive answer bias". Journal of the American Statistical Association.



Randomized Response Differential Privacy

Claim 3.5. The version of randomized response described above is $(\ln 3, 0)$ -differentially private.

Using PyMC to dig through this noisy model

- Suppose 100 students are being surveyed for cheating, and we wish to find p , the proportion of cheaters
- Since we are quite ignorant about p , we will assign it a Uniform(0,1) prior

`N = 100`

`with pm.Model() as model:`

`p = pm.Uniform("freq_cheating", 0, 1)`

Using PyMC to dig through this noisy model

- We assign Bernoulli random variables to the 100 students: 1 implies they cheated and 0 implies they did not
with model:

```
true_answers = pm.Bernoulli("truths", p, shape=N, initval=np.random.binomial(1, 0.5, N))
```
- The first coin-flip can be modeled by sampling 100 Bernoulli random variables with $p = \frac{1}{2}$: denote a 1 as a Heads and 0 a Tails
with model:

```
first_coin_flips = pm.Bernoulli("first_flips", 0.5, shape=N, initval=np.random.binomial(1, 0.5, N))
```
- Although not everyone flips a second time, we can still model the possible realization of second coin-flips:
with model:

```
second_coin_flips = pm.Bernoulli("second_flips", 0.5, shape=N, initval=np.random.binomial(1, 0.5, N))
```

Using PyMC to dig through this noisy model

- Using previous variables, we can return a possible realization of the observed proportion of "Yes" responses

with model:

```
val = first_coin_flips*true_answers + (1 - first_coin_flips)*second_coin_flips  
observed_proportion = pm.Deterministic("observed_proportion", pytensor.tensor.sum(val)/float(N))
```


Using PyMC to dig through this noisy model

- Suppose that after performing our coin-flipped interviews the researchers received 35 "Yes" responses. The researchers observe a Binomial random variable, with $N = 100$ and $p =$ `observed_proportion` with `value = 35`:

`X = 35`

```
observations = pm.Binomial("obs", N, observed_proportion, observed=True, value=X)
```

Using PyMC to dig through this noisy model

- The model:

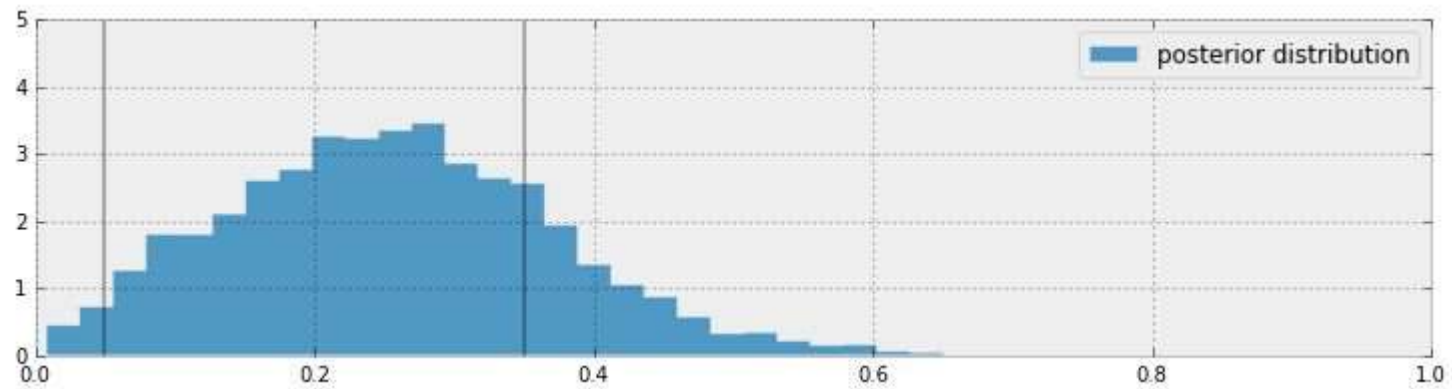
```
with model:
```

```
    step = pm.Metropolis(vars=[p])
```

```
    trace = pm.sample(25000, tune=15000, step=step, chains=1, return_inferencedata=False)
```

```
p_trace = trace["freq_cheating"]
```

Posterior Distribution



Alternative Simplified Model

Given a value for p we can find the probability the student will answer yes:

$$P(\text{Yes}) = P(\text{Heads on first coin})P(\text{cheater}) + P(\text{Tails on first coin}) P(\text{Heads on second coin}) = \frac{1}{2}p + \frac{1}{2}\frac{1}{2} = \frac{p}{2} + \frac{1}{4}$$

`N = 100`

`with pm.Model() as model:`

`p = pm.Uniform("freq_cheating", 0, 1)`

`p_skewed = pm.Deterministic("p_skewed", 0.5*p + 0.25)`

`X = 35`

`with model:`

`yes_responses = pm.Binomial("number_cheaters", 100, p_skewed, observed=35)`

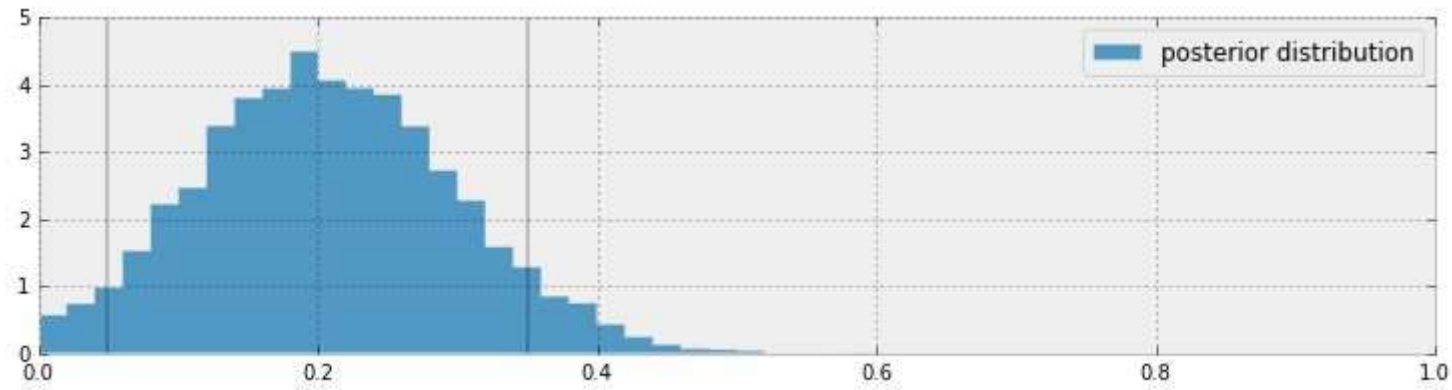
`with model:`

`step = pm.Metropolis()`

`trace = pm.sample(25000, tune=2500, step=step, chains=1, return_inferencedata=False)`

`p_trace = trace["freq_cheating"]`

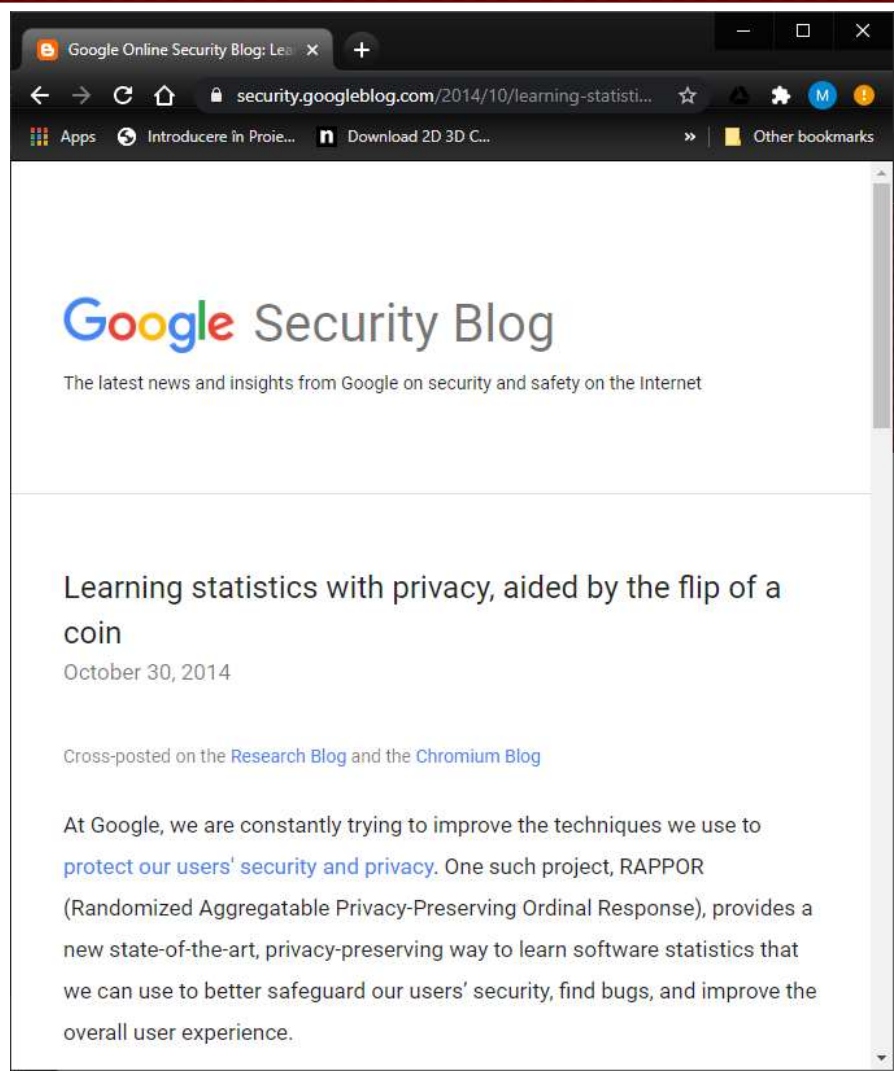
Posterior Distribution



Out of the Lab and into the Wild

Google

<https://arxiv.org/pdf/1407.6981.pdf>



RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response

Úlfar Erlingsson
Google, Inc.
ulfar@google.com

Vasyl Pihur
Google, Inc.
vpihur@google.com

Aleksandra Korolova
University of Southern California
korolova@usc.edu

ABSTRACT

Randomized Aggregatable Privacy-Preserving Ordinal Response, or RAPPOR, is a technology for crowdsourcing statistics from end-user client software, anonymously, with strong privacy guarantees. In short, RAPPORs allow the forest of client data to be studied, without permitting the possibility of looking at individual trees. By applying randomized response in a novel manner, RAPPOR provides the mechanisms for such collection as well as for efficient, high-utility analysis of the collected data. In particular, RAPPOR permits statistics to be collected on the population of client-side strings with strong privacy guarantees for each client, and without linkability of their reports.

This paper describes and motivates RAPPOR, details its differential-privacy and utility guarantees, discusses its practical deployment and properties in the face of different attack models, and, finally, gives results of its application to both synthetic and real-world data.

asked to flip a fair coin, in secret, and answer “Yes” if it comes up heads, but tell the truth otherwise (if the coin comes up tails). Using this procedure, each respondent retains very strong deniability for any “Yes” answers, since such answers are most likely attributable to the coin coming up heads; as a refinement, respondents can also choose the untruthful answer by flipping another coin in secret, and get strong deniability for both “Yes” and “No” answers.

Surveys relying on randomized response enable easy computations of accurate population statistics while preserving the privacy of the individuals. Assuming absolute compliance with the randomization protocol (an assumption that may not hold for human subjects, and can even be non-trivial for algorithmic implementations [23]), it is easy to see that in a case where both “Yes” and “No” answers can be denied (flipping two fair coins), the true number of “Yes” answers can be accurately estimated by $2(Y - 0.25)$, where Y is the proportion of “Yes” responses. In expectation, respondents will provide the true answer 75% of the time, as

The word "Apple" is written in white, sans-serif font inside a dark red speech bubble with a diagonal line pattern. The bubble has a tail pointing downwards and to the left.

Apple

https://www.apple.com/privacy/docs/Differential_Privacy_Overview.pdf

Differential Privacy

There are situations where Apple can improve the user experience by getting insight from what many of our users are doing, for example: What new words are trending and might make the most relevant suggestions? What websites have problems that could affect battery life? Which emoji are chosen most often? The challenge is that the data which could drive the answers to those questions—such as what the users type on their keyboards—is personal.